NLP-Projekt

Rino Albertin

Inhaltsverzeichnis

1 Projektbeschreibung **2**

2 Ingestion **3**

3 Vektordatenbank **4**

4 Agenten **5**

4.1 Zusammenarbeit der Agenten im RAGChatbot 5
4.2 MemoryAgent 5
4.3 RetrieverAgent 5
4.4 GeneratorAgent 6

5 Streamlit - UI **7**

6 Ausblick **9**

Abbildungsverzeichnis

1 Streamlit - UI 7
2 Beispielabfrage zum Handbuch des HAPSITE Service Moduls. 7
3 Abfrage nach der Spannung und Rückfrage von welcher Seite die Information stammt. Rechts die Seite 32. 8
4 Abfrage nach Kapitel 2 und dem Inhaltsverzeichnis. Dabei kommt es zu leichten Halluzinationen. 8
5 Abfrage nach dem Kundenservice. Nach erneuter nachfrage wird ein Link und die Seite von der die Information stammt ausgegeben. Rechts die Seite 10 mit dem Link im oberen Bereich 8

1 Projektbeschreibung

Ziel des Projekts ist die Entwicklung eines Python-basierten Chatbots, der durch die Kombination von Large Language Models (LLMs), semantischer Suche und einer Agentenarchitektur präzise Antworten auf Fragen zu PDF-Dokumenten liefert. Der Chatbot nutzt eine Retrieval-Augmented-Generation (RAG)-Architektur, die LLMs mit benutzerdefinierten Dokumenten-Embeddings kombiniert, um aktuelle und relevante Informationen bereitzustellen.

Das System besteht aus drei Agenten:

- **MemoryAgent**: Speichert und verwaltet den Kontext der Konversation.
- **RetrieverAgent**: Führt eine semantische Suche durch, um relevante Dokumente abzurufen.
- **GeneratorAgent**: Generiert Antworten basierend auf dem abgerufenen Kontext.

Benutzer können über eine Streamlit-Oberfläche PDFs hochladen und Fragen stellen, auf die der Chatbot präzise Antworten gibt. Zukünftige Erweiterungen beinhalten die Verbesserung der Speicherfunktionen und die Einführung zusätzlicher Agenten zur Optimierung des Systems.

Der Chatbot ist verfügbar unter: https://rinovative-nlp-multiagent-rag.streamlit.app/

2 Ingestion

Der Ingestion-Prozess umfasst die Extraktion und Vorverarbeitung von Daten aus PDF-Dokumenten. Zunächst werden die Metadaten des Dokuments, wie der Titel, der Autor und weitere relevante Informationen, extrahiert und im JSON-Format gespeichert. Der gesamte Text des Dokuments wird extrahiert, wobei eine erste Teilung über die y-Position an den Stellen erfolgt, an denen Absätze im Dokument vorhanden sind, da diese häufig eine natürliche logische Trennung darstellen.

Nachfolgend wird die Schriftgrösse jedes Absatzes analysiert, um die Hauptschriftgrösse zu identifizieren. Anhand der Position und der zweithäufigsten Schriftgrösse werden Kopf- und Fusszeilen sowie Überschriften und Bild- und Tabellenunterschriften detektiert. Pseudotabellen, wie z. B. Inhaltsverzeichnisse, werden ebenfalls durch die Abfolge von vielen Punkten hintereinander erkannt. Ihnen wird ein Label zugeordnet sowie das Überschrift-Level, wobei 0 die Fliesstextgrösse darstellt und grössere Grössen für Überschriften und kleinere für Bild- und Tabellenunterschriften verwendet werden. Überschriften, Tabellen, Pseudotabellen, Abbildungsunterschriften und der eigentliche Fliesstext werden in diesem Schritt ebenfalls gesammelt und in der jeweiligen Kategorie in eine Liste eingeordnet.

Nach der Extraktion und Klassifizierung der Absätze und Strukturelemente wird der Text in Chunks unterteilt. Falls ein Textabschnitt die maximale Länge überschreitet, wird er in mehrere Teile aufgeteilt. Dabei wird ein Teil des Textes des vorherigen Chunks in den nächsten übernommen, um einen fliessenden Übergang zu gewährleisten. Nach dem Chunking liegen die Daten schliesslich in folgender Form vor: Die Metadaten befinden sich zu Beginn des Dokuments in einem separaten Metadaten-Chunk. Anschliessend folgen Überschriften, Tabellen, Pseudotabellen, Bildunterschriften und Paragraphen. Jeder dieser Abschnitte erhält eine eindeutige chunk_id, die sich wie folgt zusammensetzt: `Dokumenttitel, Art (Überschrift, Tabelle, ...), Überschriftsebene, Seite, (fortlaufende Nummer falls Abschnitt geteilt wurde)`. Der Text, der an die OpenAI-API gesendet wird, hat ein ähnliches Format wie die chunk_id, gefolgt vom entsprechenden Text.

3 Vektordatenbank

Der FAISSStore ist eine Klasse, die die FAISS-Bibliothek nutzt, um Embeddings von Texten zu speichern und zu durchsuchen. Zunächst wird ein FAISS-Index erstellt oder geladen, wenn er bereits existiert. Die Klasse extrahiert Embeddings aus den Dokumenten und ordnet jedem Textabschnitt die eindeutige `chunk_id` zu. Diese Embeddings werden im FAISS-Index gespeichert, sodass sie effizient durchsucht werden können.

Für die Suche werden Vektoren abgefragt, und die k nächsten Treffer werden basierend auf ihrer semantischen Ähnlichkeit zurückgegeben. Mit der Methode `search()` können die Treffer einschließlich der `chunk_id`, der Distanz und des zugehörigen Texts abgerufen werden. Zusätzlich können Texte anhand ihrer `chunk_id` schnell abgerufen werden.

Der FAISSStore bietet so eine effiziente Möglichkeit, semantische Suchen auf großen Textmengen durchzuführen, die in Vektoren umgewandelt wurden.

ß

4 Agenten

4.1 Zusammenarbeit der Agenten im RAGChatbot

Die Agenten im **RAGChatbot** arbeiten zusammen, um die Benutzeranfrage zu verarbeiten und eine präzise Antwort zu generieren. Der Ablauf erfolgt in einem **StateGraph**, der die Interaktionen zwischen den Agenten steuert.

- **MemoryAgent**: Der **MemoryAgent** ist verantwortlich für das Abrufen und Speichern des Kontexts. Zu Beginn wird der Kontext der letzten Nachricht des Benutzers aus dem Speicher abgerufen. Der Agent fügt dann die neue Benutzeranfrage zum Speicher hinzu, um eine kontinuierliche Interaktion zu gewährleisten.
- **RetrieverAgent**: Der **RetrieverAgent** holt relevante Dokumente oder Abschnitte aus der Datenbank, die auf der Benutzeranfrage und dem gespeicherten Chatverlauf basieren. Dieser Schritt ermöglicht es, den benötigten Kontext aus externen Quellen zu extrahieren, um eine fundierte Antwort zu generieren.
- **GeneratorAgent**: Der **GeneratorAgent** nutzt den abgerufenen Kontext sowie den gespeicherten Chatverlauf, um eine Antwort zu generieren. Er verarbeitet die Benutzeranfrage zusammen mit dem Kontext und gibt eine entsprechende Antwort zurück.
- **Erneute Speicherung durch MemoryAgent**: Nach der Antwortgenerierung wird der **MemoryAgent** erneut verwendet, um den neuen Kontext sowie die generierte Antwort zu speichern, damit zukünftige Interaktionen auf diesem aktualisierten Wissen basieren.

Der Ablauf ist durch den **StateGraph** organisiert, der sicherstellt, dass jeder Agent seine Aufgabe in einer definierten Reihenfolge ausführt. Der **MemoryAgent** speichert und verwaltet den Kontext, der **RetrieverAgent** holt relevante Informationen, und der **GeneratorAgent** erstellt die Antwort. Durch diese Zusammenarbeit der Agenten wird eine kontextuell relevante und kohärente Antwort auf die Benutzeranfrage generiert.

4.2 MemoryAgent

Der MemoryAgent speichert den Gesprächskontext, indem er Benutzeranfragen und Assistentenantworten in Redis ablegt. Mit der Methode `add_to_memory()` wird jede neue Nachricht gespeichert, wobei der Verlauf auf eine maximale Anzahl (`max_history`) begrenzt wird. Die Methode `get_memory()` ruft den relevanten Kontext ab, einschliesslich aller Benutzeranfragen und der letzten Assistentenantwort. Ursprünglich nutzte der Agent das Modell `SentenceTransformer` zur semantischen Analyse, um zu bestimmen, welcher Kontext geladen werden soll. Diese Methode wurde jedoch vorerst verworfen, da sie in einigen Fällen zu unerwünschten Schleifen führte und die Effizienz beeinträchtigte.

4.3 RetrieverAgent

Der **RetrieverAgent** ist dafür verantwortlich, relevante Dokumente basierend auf einer Benutzeranfrage aus der FAISS-Datenbank abzurufen. Der Agent arbeitet in mehreren Schritten:

- **Semantische Suche**: Der Agent empfängt eine Benutzeranfrage und kombiniert diese mit dem Gedächtnis (Kontext), das vom **MemoryAgent** bereitgestellt wird. Anschliessend wird ein Embedding der kombinierten Anfrage erstellt.
- **Embeddings und Suche**: Die Anfrage wird mit Hilfe von `SentenceTransformer` und `OpenAI` in einen Vektor (Embedding) umgewandelt. Dieser Vektor wird dann verwendet, um die semantische Suche in der FAISS-Datenbank durchzuführen. Je nach Detailgrad der Anfrage (`high`, `medium`, `low`, etc.) wird eine unterschiedliche Anzahl von Ergebnissen (Top-k) abgerufen.
- **Detaillierungsgrad**: Der Detailgrad wird durch die Methode `_should_be_detailed()` bestimmt. Diese bewertet die Anfrage, um zu entscheiden, ob eine detaillierte oder eher allgemeine Antwort erforderlich ist. Dies geschieht anhand der `Kosinusähnlichkeit` zwischen der Anfrage und vordefinierten Kategorien.
- **Ergebnisse zusammenstellen**: Der Agent extrahiert den relevanten Text aus den abgerufenen Dokumenten und stellt ihn als Kontext zur Verfügung.

Zusammengefasst sorgt der **RetrieverAgent** dafür, dass die semantisch passenden Dokumente abgerufen werden, basierend auf der Benutzeranfrage und dem vorherigen Kontext, wodurch eine präzise und kontextbezogene Antwort ermöglicht wird.

4.4 GeneratorAgent

Der **GeneratorAgent** ist dafür verantwortlich, auf die Benutzeranfragen eine Antwort zu generieren, indem er den Kontext und das Gedächtnis des Benutzers einbezieht. Die Methode `generate_answer()` erstellt einen Prompt, der die Benutzeranfrage sowie den abgerufenen Kontext und das Gedächtnis enthält, falls diese vorhanden sind. Der Prompt wird wie folgt aufgebaut:

```
Frage:
{Benutzeranfrage}

Beantworte diese Frage basierend auf den relevanten Dokumenten:
{Kontext}

Erinnere dich dabei an:
{Gedächtnis}
```

Auf dieser Grundlage wird die Antwort generiert, die dann an den Benutzer zurückgegeben wird. So stellt der **GeneratorAgent** sicher, dass der generierte Text präzise, informativ und im Kontext der Anfrage relevant ist.

5 Streamlit - UI

Abbildung 1: Streamlit - UI

Die Streamlit-Oberfläche ermöglicht eine benutzerfreundliche Interaktion mit dem PDF-basierten Chatbot. Nutzer können mehrere PDFs hochladen, jedoch wird aufgrund eines Fehlers momentan nur das erste Dokument korrekt verarbeitet. Alle Dateien werden in der Session des Benutzers gespeichert. Nach dem Hochladen starten die Dokumente mit der Extraktion und Analyse des Inhalts. Der Benutzer kann anschliessend eine Frage zum hochgeladenen Dokument stellen, die in ein Textfeld eingegeben wird. Nach der Verarbeitung wird die Antwort auf der Benutzeroberfläche angezeigt. Diese Funktionalität erlaubt es den Nutzern, interaktiv mit den hochgeladenen Dokumenten zu arbeiten, um gezielte Informationen zu extrahieren und präzise Antworten zu erhalten. Tests wurden unter anderem mit dem Handbuch des HAPSITE Service Moduls durchgeführt, das für die Bereitstellung von Vakuum und Notstromversorgung im HAPSITE Chemical Identification System verantwortlich ist. Nachfolgend einige Beispiel dazu.

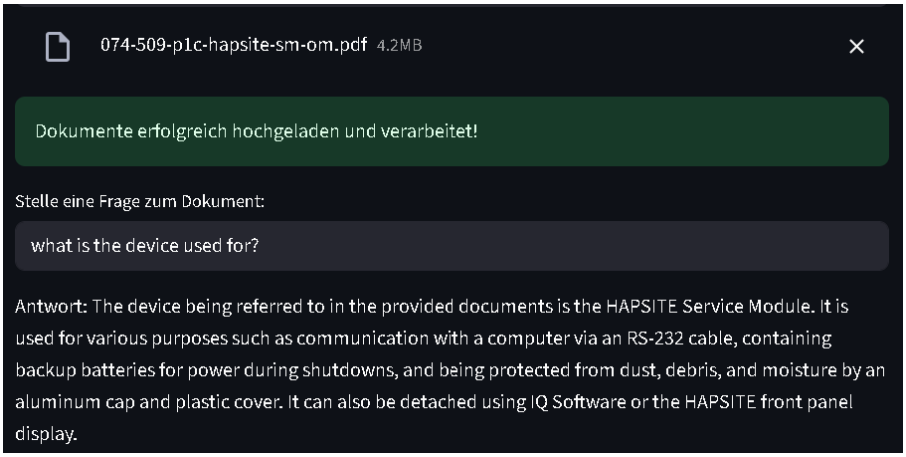

Abbildung 2: Beispielabfrage zum Handbuch des HAPSITE Service Moduls.

what is the voltage

Antwort: Based on the relevant documents provided, the voltage for the HAPSITE device is approximately 24 V (dc).

give me the tabel from which you got that information

Antwort: I apologize for the confusion. The information regarding the voltage for the HAPSITE device is based on the relevant documents provided from the "HAPSITE - Pseudotable" on page 32, where it states that the Supply Volts and Battery both are approximately 24 V (dc).

INFICON — *HAPSITE Service Module Operating Manual*

The following status items are reported in the **Service Module** tab of the **Properties** Window. Refer to Figure 5-9.

Supply Volts (V) . . . . . Approximately 24 V (dc)

Battery (V) . . . . . Approximately 24 V (dc)

Battery Level (%) . . . . . The charge percentage of the HAPSITE battery (inserted in the battery compartment of the Service Module), as a percentage of the full capacity. Displays "No Battery" if a battery is not installed.

Abbildung 3: Abfrage nach der Spannung und Rückfrage von welcher Seite die Information stammt. Rechts die Seite 32.

Stelle eine Frage zum Dokument:

what is chapter 2 about?

Antwort: Chapter 2 of the document is about specifications related to the HAPSITE device. It includes details such as physical specifications and operating specifications.

give me the tabel of contents

Antwort: Based on the relevant documents, the Table of Contents for the HAPSITE document is as follows:

1. Introduction
2. Specifications
3. Installation
4. Operation
5. Maintenance
6. Troubleshooting
7. Appendix

If you have any more questions or need further assistance, feel free to ask.

Chapter 1

Manual Use and General Safety

1.1 General Safety 1-1
1.2 Using This Manual 1-2
1.2.1 Symbols and Their Definitions 1-2
1.3 How To Contact INFICON 1-3
1.3.1 Returning the HAPSITE Service Module 1-3

Chapter 2

Specifications

2.1 Physical Specifications. 2-1
2.2 Operating Specifications 2-1

Chapter 3

Overview

3.1 Introduction. 3-1
3.2 Service Module Component Overview. 3-2
3.2.1 UPS (Backup) Batteries 3-4
3.2.2 Load Lock Assembly 3-5
3.3 Communications. 3-6
3.4 Service Module Power Input 3-7
3.4.1 AC Power Input 3-7
3.4.2 24 V (dc) Input Supply 3-7

Chapter 4

Service Module Operation

4.1 When to Use the Service Module. 4-1
4.2 Using the Service Module 4-2
4.2.1 Setting Up the Service Module. 4-2

Abbildung 4: Abfrage nach Kapitel 2 und dem Inhaltsverzeichnis. Dabei kommt es zu leichten Halluzinationen.

how do i get customer service

Antwort: Based on the information provided in the relevant documents for the HAPSITE, if you need to contact customer service, you can follow these steps:

1. Contact Sales and Customer Service to find the nearest INFICON Sales office. This information is available in the document.
2. Before returning any component of the HAPSITE Service Module to INFICON, make sure to speak with a Customer Support Representative first and obtain a Return Material Authorization (RMA) number.
3. If you have questions regarding troubleshooting, diagnosing, repairing a defective HAPSITE Service Module, or if you require technical support for applications and programming, you can reach out to a Service Engineer or Technical Support Engineer respectively.

If you have any specific questions or need further assistance, feel free to ask.

at which pages do i find these informations?

Antwort: Based on the information provided in the relevant documents for the HAPSITE, you can find the information you are looking for on the following pages:

1. The first piece of information regarding returning any component of the HAPSITE Service Module to INFICON can be found on Page 10, under Heading Level 0 in the HAPSITE document.
2. The second piece of information about worldwide customer support information being available under Support at www.inficon.com can also be found on Page 10, under Heading Level 0 in the HAPSITE document.

Therefore, you can refer to Page 10 of the HAPSITE document to find both pieces of information related to contacting customer service and returning components of the HAPSITE Service Module to INFICON.

INFICON — *HAPSITE Service Module Operating Manual*

1.3 How To Contact INFICON

Worldwide customer support information is available under Support at www.inficon.com to contact:

- a Technical Support Engineer with questions regarding applications for and programming the HAPSITE
- a Service Engineer with questions regarding troubleshooting, diagnosing or repairing a defective HAPSITE Service Module
- Sales and Customer Service, to find the nearest INFICON Sales office
- Repair Service, to find the nearest INFICON Service Center

If experiencing a problem with the HAPSITE Service Module, please have the following information readily available:

- the serial number and firmware version for the HAPSITE Service Module
- a description of the problem
- an explanation of any corrective action already been attempted
- and the exact wording of any error messages received

1.3.1 Returning the HAPSITE Service Module

Do not return any component of the HAPSITE Service Module to INFICON without first speaking with a Customer Support Representative and obtaining a Return Material Authorization (RMA) number.

Packages to INFICON without an RMA number will be held until the customer is contacted. This will result in delays in servicing the HAPSITE Service Module.

Abbildung 5: Abfrage nach dem Kundenservice. Nach erneuter nachfrage wird ein Link und die Seite von der die Information stammt ausgegeben. Rechts die Seite 10 mit dem Link im oberen Bereich

Zusammenfassend liefert das Modell keine schlechten Ergebnisse, jedoch kommt es teilweise zu Halluzinationen oder Wiederholungen. Diese beiden Punkte deuten darauf hin, dass der Memory-Agent momentan noch nicht optimal arbeitet. Das Modell kann unter folgendem Link selbstständig getestet werden: https://rinovative-nlp-multiagent-rag.streamlit.app/

6 Ausblick

Zukünftige Arbeitsschritte beinhalten die Erweiterung des Dateimanagements, da aktuell das System nur ein Dokument gleichzeitig verarbeiten kann. Um die Bearbeitung mehrerer Dokumente zu ermöglichen, muss die Datenstruktur überarbeitet und die Dokumentenverwaltung angepasst werden. Ein weiteres Ziel ist dabei, die Möglichkeit zu schaffen, mehr Dokumenttypen einlesen zu können.

Ein weiteres wichtiges Ziel ist die Verbesserung des MemoryAgents, der derzeit noch ungenügend arbeitet. Es müssen Speicherung und Abruf von Kontextinformationen gezielter gestaltet werden, um eine flüssigere und kontextuell konsistente Interaktion zu gewährleisten. Hier könnte ein ManagementAgent hilfreich sein, der die verschiedenen Agenten besser koordiniert und eine strukturierte Verwaltung des Gesprächsverlaufs ermöglicht.

Darüber hinaus gibt es mehrere zukünftige Agenten, die das System weiter verbessern könnten. Ein ClassifierAgent könnte eingeführt werden, um zwischen verschiedenen Dokumenttypen und Anfragetypen (semantisch oder exakt) zu unterscheiden und entsprechend die richtigen Retrieval-Strategien auszuwählen. Der RetrieverAgent könnte weiter spezialisiert bzw. in mehrere Agenten aufgeteilt werden, um beispielsweise Tabellen, Bildunterschriften, Seiteninhalte, Links, Metadaten oder Inhaltsverzeichnisse gezielt zu extrahieren. Ein weiterer Agent könnte speziell dafür zuständig sein, Inhalte zusammenzufassen. Auch ein FeedbackAgent wäre denkbar, der Rückfragen stellt, falls die Antwort des Systems unklar oder unvollständig ist. Ein UserExpectationClassifierAgent könnte dabei helfen, die generierte Antwort mit den Erwartungen des Benutzers abzugleichen, um sicherzustellen, dass die Antwort wirklich den gewünschten Inhalt bietet.

Durch diese Erweiterungen wäre das System deutlich flexibler und würde robuster sowie leistungsfähiger werden.